

14 APRIL

2026

Queueing Theory in Practice: Empirical Study of a Microservice

SOFTWARE PERFORMANCE

ENGINEERING

25/26

PROFESSOR

MIRCO TRIBASTONE

GROUP

KEVIN SHEMILI

DARIO PIPA

CONTENTS

1.	INTRODUCTION	3
2.	ARCHITECTURE & TOPOLOGY	4
2.1.	μBench Overview	4
2.2.	Work Model	4
2.3.	Workload Generation	5
2.4.	Monitoring	7
2.4.1.	Prometheus Scrapper	8
2.4.2.	Jaeger Scrapper	9
3.	EMPIRICAL RESULTS	11
3.1.	ASSUMPTION 1: Poisson Arrival Process	12
3.1.1.	Generated Workload Arrivals	12
3.1.2.	Runner's Realized Arrivals	14
3.1.3.	Arrival Processes at Internal Services	15
3.2.	Assumption 2: Exponential Service Times	19
3.2.1.	Internal Service-Time Distribution	19
3.2.2.	Sojourn-Time Distribution per Service	21
3.2.3.	Service-Time Decomposition: Internal vs External	25
3.3.	Assumption 3: Single Server Behavior	26
3.4.	Assumption 4: Infinite Buffer	27
3.5.	Further Considerations	28
3.5.1.	Little's Law	28
3.5.2.	Visit Ratio Validation	30
3.5.3.	Bottleneck Identification	31
3.5.4.	E2E Analysis	33
4.	CONCLUSION	35
5.	REFERENCES	36

1. INTRODUCTION

During our “Software Performance Engineering” course, we saw performance modeling techniques based on stochastic processes, things like Markov models, Poisson arrivals, exponential service times, and classical queueing systems such as M/M/1 and M/M/c. These models are useful because they make performance analysis mathematically feasible, but in real systems their assumptions do not always hold up. This statement is especially relevant in something like a microservice system, where request flows can become bursty, services correlated & downstream calls can introduce waiting dependencies that are not captured well by simple analytical models.

Through this project, we aim to investigate how well these theoretical assumptions align with the practical behavior of a real microservice system. To achieve this, we designed and conducted a series of experiments in which we generated synthetic workloads, collected traces & performance metrics, and analyzed them using a statistical approach. So, the focus is on understanding if fundamental queueing assumptions are valid in practice, and how deviations from these assumptions affect system behavior.

For this very purpose, we have used the μ Bench project^[1] as the benchmark for our experiments. μ Bench, also proposed in the project’s requirements list, is a synthetic microservice benchmark that allows for the definition of custom service topologies, workload characteristics, and resource configurations. We went in favor of μ Bench, instead of something like Google Boutique or Sock Shop due to its high degree of configurability & very good documentation, especially the later which allowed us to be able to very easily deploy the application & set it up almost straight away.

2.ARCHITECTURE & TOPOLOGY

2.1. μ Bench Overview

As previously stated, μ Bench is the application which we are going to use as the benchmark for our analysis. By configuring it, μ Bench can generate applications composed of multiple interacting services, where each service executes some local computation & may invoke other services to complete a request.

Without getting into too much detail, the main components of μ Bench can be summarized as follows:

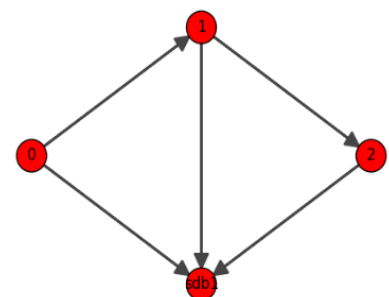
- **Service Cells:** Each service is implemented as a containerized unit that performs some internal computation and may call other services.
- **Work Model:** Central file that defines the behavior of each service, including downstream calls.
- **Workload Generation:** Can be generated with specific statistical properties.
- **Monitoring & Tracing:** In-built Integration with Prometheus & Jaeger.

2.2. Work Model

The microservice's behavior is fully defined in this file. The work model is not created by hand, rather we have followed the μ Bench's workflow where first we create a service dependency graph, which determines which services call each other and then enrich this graph with internal service functions & resource configurations.

This process has yielded the resulting topology, which consists of four services, where s0 acts as the entry point of the system. Each service is configured with 1 replica and 4 workers, forming an M/M/4 queue at every node.

- **S0**
 - Main entry point, that a request first encounters.
 - Internally uses the colosseum function, an extremely lightweight computation.
 - Interacts in parallel with s1 & sdb1.



- **S1**
 - Receives requests from s0.
 - Internally performs artificial computation to simulate processing time.
 - Interacts in parallel with s2 & sdb1.
- **S2**
 - Downstream service called by s1.
 - Internally performs artificial computation to simulate processing time.
 - Calls sdb1.
- **SDB1**
 - Shared database component, accessed by multiple services.
 - Internally instead of CPU heavy computation, it causes disk stress by performing file I/O operations.
 - Does not call any further services.

2.3. Workload Generation

For generating the traffic in order to be able to analyze the system, we have used the built-in load generator of μ Bench. It is comprised out of two components:

- **Traffic Generator:** Produces a workload file containing timestamped request arrivals based on a specified statistical distribution.
- **Runner:** Consumes file generated by Traffic Generator & thus replays requests against the deployed system while recording execution data such as timestamps, response times, and status codes.

Traffic Generator samples inter-arrival times from an exponential distribution, which results in a Poisson arrival process. Each workload file contains 5000 request arrivals, which provides a sufficient sample size for the subsequent statistical hypothesis tests. For our analysis to be meaningful, we evaluate the system under 3 different load conditions:

- **Low Load:** Mean Interarrival Time 500ms
 - $\lambda = \frac{1}{500 \text{ (ms)}} = 2$
- **Medium Load:** Mean Interarrival Time 200ms
 - $\lambda = \frac{1}{200 \text{ (ms)}} = 5$
- **High Load:** Mean Interarrival Time 100ms
 - $\lambda = \frac{1}{100 \text{ (ms)}} = 10$

We have though it best to use Poisson traffic for this experiment. As we know classical queueing models typically assume that arrivals follow a Poisson process, which implies exponentially distributed inter-arrival times & independence between arrivals. By generating traffic that satisfies these assumptions at the input of the system, we ensure that any observed deviations in behavior are introduced by the system itself rather than by the workload.

To further guarantee this, the Runner is configured with a thread pool of 500 concurrent threads, ensuring it never becomes the bottleneck. It can dispatch requests at the full rate prescribed by the pre-generated Poisson sequence without introducing artificial queuing delays.

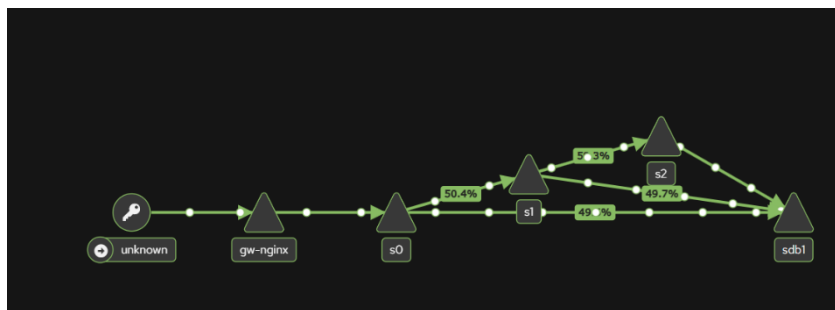
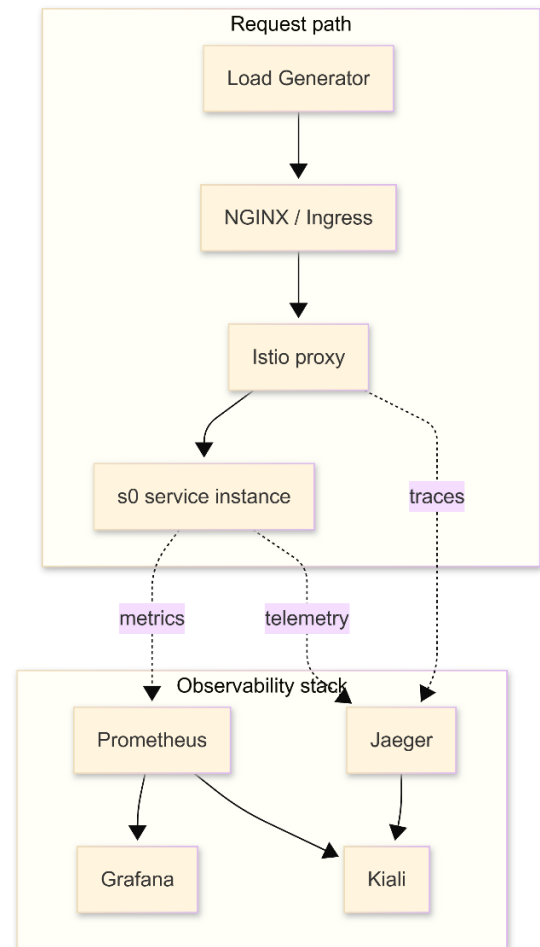
The Runner produces an output in the form of a text file, where it writes its results. Each line corresponds to a single request and for each request it records five features (columns). Captured fields are described in the table below:

<i>COLUMN</i>	<i>MEANING</i>
<i>now_ms (1st)</i>	Timestamp at which the request was dispatched
<i>req_latency_ms (2nd)</i>	Round-trip latency, so measured time from dispatch to response
<i>status_code (3rd)</i>	HTTP status code returned by the service
<i>processed_requests.value (4th)</i>	Cumulative count of requests completed at logging time
<i>pending_requests.value (5th)</i>	Number of requests currently in-flight at the moment of logging

2.4. Monitoring

For obtaining the relevant KPIs of this project we have used the μ Bench's built-in monitoring stack. It consists of the following components:

- **Prometheus:** Scrapes metrics exported by service cells every 30s.
- **Grafana:** Visualization layer on top of Prometheus.
- **Istio:** Sits in the communication path between services through proxies & enables deeper observability.
- **Jaeger:** Collects request traces across microservice chain, allowing us to reconstruct end-to-end request flows and measure per service span durations.
- **Kiali:** Graphical view of interactions between services as observed through Istio.



2.4.1. Prometheus Scrapper

For metric collection, we queried Prometheus directly over the time interval corresponding to experiment's run. By default, μ Bench service-cells export application-level metrics to Prometheus^[2], which makes it possible to observe not only E2E request latency, but also the internal and external parts of service execution separately.

The metrics scraped in our experiments are the following:

- **“mub_internal_processing_latency_milliseconds”**: Metrics capture duration of the internal-service processing only, so actual work performed locally. Important as it allows us to approximate service's own computation time without downstream waiting. By taking their ratio over some interval, we get the average internal processing latency.
 - **_count**: Observed number.
 - **_sum**: Cumulative time in milliseconds.
 - **_bucket_bucket**: Cumulative count of internal latencies up to each bucket threshold.
- **“mub_request_processing_latency_milliseconds”**: Overall request latency seen at a service, so total time spent handling a request, including both the service's own internal work and any waiting caused by downstream calls.
 - **_count**: Observed number
 - **_sum**: Cumulative time in milliseconds.
 - **_bucket_bucket**: Cumulative count of request latencies.
- **“mub_external_processing_latency_milliseconds”**: Capture time spent in external-service execution, so time associated with downstream calls.
 - **_count**: Observed number.
 - **_sum**: Cumulative time in milliseconds.
 - **_bucket_bucket**: Cumulative count of external-processing observations.

For collection, the query was restricted to the actual time period during which the experiment was conducted. This was achieved by reading the **“now_ms”** column from the Runner's text file to determine when each request was dispatched, and the **“req_latency_ms”** column to determine when each request completed. The earliest **“now_ms”** value becomes the left edge of the query window, while the latest computed completion (**“now_ms + req_latency_ms”**) timestamp becomes the right edge. A buffer of 30 seconds is then added on both sides to guard against clipping any metric activity at the boundaries.

In the collector we find 2 core functions:

- **“query_instant(promql, ts)”**: Execute a PromQL query at a given timestamp.
 - This is used to retrieve the total accumulated value of a metric over the experiment window. To do this, we use the `increase()` PromQL function, which computes how much a counter grew over a specified duration.
 - Resulting query has following form:

```
increase(mub_request_processing_latency_milliseconds_count{app_name="s0"}[<window_s>s])
```

- **“query_range_raw(metric, service, start_s, end_s)”**: Execute a PromQL query for the full time series of a metric between start & end of the experiment, returning one data point every 30 seconds.
 - Applied to the **“_bucket_bucket”** metrics, for retrieving the full time series rather than a single final total, which is necessary for distributional analysis.

```
mub_internal_processing_latency_milliseconds_bucket_bucket{app_name="s0"}
```

After the run, the scraper produces a single JSON file, with the following structure:

KEY	CONTENT
<i>window</i>	Time window of experiment
<i>increase</i>	For each metric & each service, total amount by which that metric grew
<i>raw_bucket_series</i>	For each <code>_bucket_bucket</code> metric & each service, full time series of raw counter values sampled every 30 seconds

2.4.2. Jaeger Scraper

Jaeger gives us the means to observe per-request execution traces. To collect traces, Jaeger was queried separately for 4 deployed services. The information extracted in our experiments is the following:

- **“traceID”**: Identifies full trace to which a span belongs, so it links one service’s activity to the E2E request path across microservice.
- **“spanID”**: Identifies individual span within a trace, allowing each service execution instance to be uniquely distinguished.

- **“start_us”**: Timestamp at which span started.
- **“duration_us”**: Duration of span.

As was the case with Prometheus, Jaeger queries were restricted to the time interval corresponding to actual workload execution, derived from result file produced by Runner. A buffer of 30 seconds was again added on both sides of this interval to ensure that spans starting slightly before or finishing slightly after the exact request window are still captured.

In this collector we find 2 core functions:

- **“fetch_traces(service, start_us, end_us)”**: Query Jaeger for all traces involving a given service within some experiment window.
 - We issue the request to the /api/traces endpoint, thus the call takes the following form for each service:

```
GET
/jaeger/api/traces
?service=s0.default
&start=<start_us>
&end=<end_us>
&limit=100000
```

- **“extract_spans(traces, target_service)”**: Process traces returned by the API & filter them by:
 - Service name: So, match only services that match target
 - Span kind: Only server-side spans are kept, to ensure that work is counted only once.
 - There is a further span kind, known as the client one, which occurs when there is a downstream call. So, a call entails creation of a client span on the calling side & a server span on the receiving side. Retaining both would mean the double counting of latency already captured by the downstream service's own server span.

The scraper produces the following JSON file:

<i>FIELD</i>	<i>CONTENT</i>
<i>Key</i>	Service name (s0.default, s1.default...)
<i>traceID</i>	Identifier of the full distributed trace span belongs to
<i>spanID</i>	Identifier of individual span

<i>start_us</i>	Span start timestamp
<i>duration_us</i>	Span duration

3. EMPIRICAL RESULTS

The empirical analysis of our experiment has been put in a single Jupyter Notebook, which serves as the main deliverable for this project. Here we make use of all 4 data sources produced by the collection pipeline: 1. Runner result file, 2. Prometheus JSON file, 3. Jaeger JSON file, 4. Scheduled requests by Traffic Generator.

```
RUNNER_RESULT_FILE = "../../SimulationWorkspace/Result/runner_high_results.txt"
WORKLOAD_FILE = "../../SimulationWorkspace/high_load.json"

JAEGER_FILE = "../../spe-project/data/jaeger/spans_high.json"
PROMETHEUS_FILE = "../../spe-project/data/prometheus/prometheus_high.json"
```

The analysis furthermore makes of the following important 2 helper functions:

- **“prometheus_mean(data, metric_count, metric_sum, service)”**: Compute mean for a given metric and service by retrieving the increase() values for both the count and sum fields from the Prometheus JSON file and computing:

$$\bar{x} = \frac{\text{total_sum}}{\text{total_count}}$$

- **“reconstruct_samples_from_histogram_buckets(bucket_results)”**: Several analyses conducted require a flat array of individual latency samples. Prometheus histograms do not store individual observations, they instead store cumulative bucket counters, where each bucket records how many observations have fallen below a given latency threshold. What we do here is to reconstruct an approximate array of individual samples from those bucket counters through the following steps:
 - **Remove +Inf bucket**, as it has no finite upper bound and cannot be represented as a midpoint.
 - **Isolate per-bucket counts** by subtracting each bucket's cumulative series from the one below it, recovering how many observations fell strictly within each bucket interval.
 - **Difference over time** by applying np.diff to each bucket's cumulative series across scrape intervals.
 - **Approximate each observation** by the midpoint of its bucket interval ((lower_le + upper_le) / 2).

3.1. ASSUMPTION 1: Poisson Arrival Process

As we have seen during classes, classical queueing theory assumes that requests arrive according to a Poisson process, which implies that inter-arrival times are exponentially distributed and independent over time. This assumption is central to models such as M/M/1, as it ensures memoryless arrivals & simplifies analytical treatment of the system.

In this section, we aim to verify whether this assumption holds in practice within our microservice setup.

3.1.1. Generated Workload Arrivals

In this part of the notebook, we verify whether the workload generated by μ Bench satisfies the arrival assumption that classical queueing models expect at the input of the system.

This should implicitly pass, since the Traffic Generator is configured to sample inter-arrival times from an exponential distribution, but regardless this is a safe check to ensure that any possible downstream discrepancies are not caused by a problem at this level.

At a high level, in the code we:

- Read arrival timestamps from the generated “workload.json” file, which contains the request executed by Runner.
- Compute inter-arrival times by taking differences between consecutive arrivals.
- Compute basic descriptive statistics, importantly the coefficient of variation (CV), where a value close to 1 is consistent with exponential inter-arrival times.
- Fit an exponential distribution to the empirical inter-arrival times & then apply a Kolmogorov–Smirnov on a capped random sample of 500 observations to evaluate whether the observed data is statistically compatible with fitted exponential. Cap is important as KS is sensitive to large sample sizes, detect negligible deviations as statistically significant.

We obtain the following results (high load):

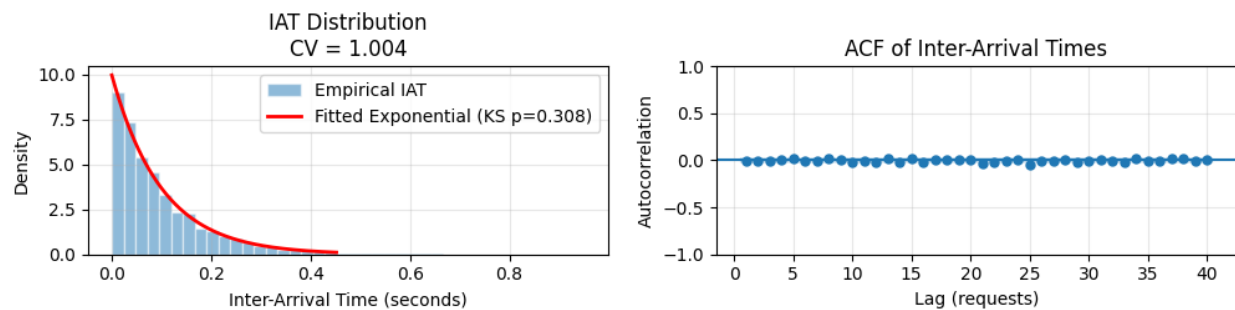
METRIC	VALUE
Mean IAT	0.0996 s

<i>Std IAT</i>	0.1000 s
<i>CV</i>	1.0038
<i>KS Statistic</i>	0.0429
<i>KS p-value</i>	0.3084
<i>Verdict</i>	PASS (cannot reject exponential)

Results confirm that the Traffic Generator produces arrivals that satisfy the theoretical properties of a Poisson process. Furthermore, results hold consistently across all 3 load levels:

- CV is very close to 1 in all cases, the expected value for an exponential distribution where mean & std are equal.
- KS test p-values are high (>0.05) across all load levels, providing no statistical evidence to reject the hypothesis.

Assumption 1 : Poisson Arrivals at s0 (Traffic Generator)



HIGH LOAD

For high load specifically:

- Histogram shows a right-skewed distribution with many small inter-arrival times and a long tail, closely resembling expected shape of an exponential distribution.
- Autocorrelation values fluctuate randomly around zero across all lags without any visible structure, confirming that successive inter-arrival times are independent.

Overall expected results, since Traffic Generator is explicitly configured to sample inter-arrival times from an exponential distribution.

3.1.2.Runner's Realized Arrivals

In this step, we check whether the arrival pattern produced by the Traffic Generator, is correctly realized by the Runner during execution. Specifically, whether timestamps at which the Runner dispatches requests follow a Poisson process, or whether execution effects such as scheduling jitter or runtime overhead introduce distortions.

At a high level, in the code we:

- Read request-dispatch timestamps recorded by Runner from the result file, which represent the timestamps at which the Runner dispatched requests.
- Compute inter-arrival times by taking differences between consecutive timestamps.
- Calculate basic descriptive statistics of the realized inter-arrival sequence, most importantly the CV.
- Fit exponential distribution to realized inter-arrival times & apply KS test to compare the empirical inter-arrival distribution against the fitted exponential model.

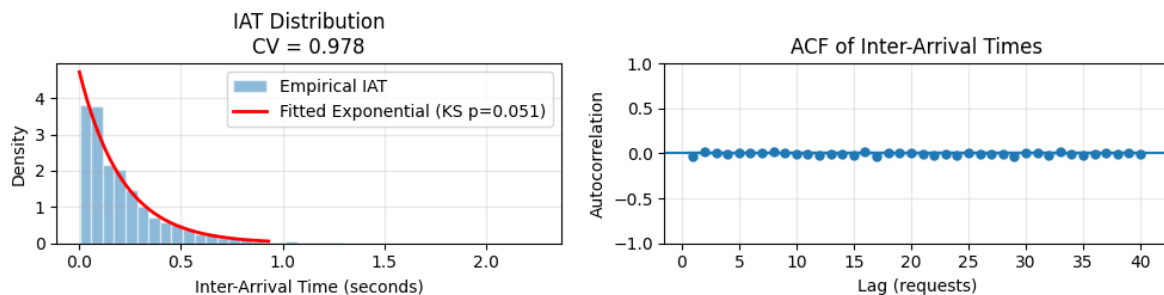
We obtain the following results (high load):

<i>METRIC</i>	<i>VALUE</i>
<i>Mean IAT</i>	0.0999 s
<i>Std IAT</i>	0.1000 s
<i>CV</i>	1.0011
<i>KS Statistic</i>	0.0395
<i>KS p-value</i>	0.4056
<i>Verdict</i>	PASS (cannot reject exponential)

Realized process by the Runner preserves statistical properties of the Poisson process, and this is true for all 3 load levels.

- CV remains equal to 1 across all load levels, confirming that the exponential character of the inter-arrival distribution.
- KS p-values are high (>0.05) in all cases, providing no statistical evidence to reject the exponential hypothesis.
- Mean IAT is nearly identical to the configured value (~ 0.1 s high load), confirming the Runner dispatches requests at the intended rate without delay.

Assumption 1 : Poisson Arrivals at s0 (Realized)



HIGH LOAD

These results also directly validate the design decision to configure the Runner with a large thread pool, ensuring it does not introduce any kind of overhead effects itself.

Therefore, we have an agreement between generated & realized inter-arrival distributions, confirming that the Runner introduces no measurable distortions.

3.1.3. Arrival Processes at Internal Services

Now we get into the actual detailed checks, where we examine whether the Poisson property is preserved as requests propagate through the internal services of the application. This is the most realistic, since it evaluates the arrivals that the system actually sees rather than the planned ones.

The main idea here is to test whether the arrival streams observed at s0, s1, s2 & sdb1 still behave like Poisson processes, since under the assumptions of an open Jackson network, internal arrival processes are also expected to remain Poisson. This makes the internal-service

analysis important, because even if the ingress stream is well behaved, routing, parallel calls, synchronization & downstream dependencies may still distort arrivals inside the network.

At a high level, in the code we:

- Iterate over all services in topology & extracts span start timestamps, collected from Jaeger, to compute inter-arrival times by taking consecutive differences.
- Compute basic statistics for resulting inter-arrival sequence, including mean, std & CV.
- Fit an exponential distribution to the empirical inter-arrival times & apply KS test to compare empirical inter-arrival data against the fitted exponential model.

We obtain the following results (across all loads):

<i>SERVICE</i>	<i>SPANS</i>	<i>MEAN IAT</i>	<i>CV</i>	<i>KS STATISTIC</i>	<i>KS P-VALUE</i>	<i>VERDICT</i>
LOW LOAD						
<i>s0.default</i>	5000	0.5016	0.965	0.0425	0.3179	PASS
<i>s1.default</i>	5000	0.5012	0.948	0.0430	0.3036	PASS
<i>s2.default</i>	5000	0.5011	0.908	0.0902	0.0005	FAIL
<i>sdb1.default</i>	15000	0.1670	1.336	0.1413	0.0000	FAIL
MEDIUM LOAD						
<i>s0.default</i>	5000	0.2032	1.053	0.1508	0.0000	FAIL
<i>s1.default</i>	5000	0.2067	0.518	0.2443	0.0000	FAIL
<i>s2.default</i>	5000	0.2067	0.521	0.2309	0.0000	FAIL
<i>sdb1.default</i>	15000	0.0689	0.932	0.0592	0.0575	PASS
HIGH LOAD						
<i>s0.default</i>	2176	0.2397	1.703	0.6065	0.0000	FAIL
<i>s1.default</i>	1935	0.2692	0.568	0.1934	0.0000	FAIL
<i>s2.default</i>	1933	0.2694	0.579	0.2048	0.0000	FAIL
<i>sdb1.default</i>	5805	0.0899	0.978	0.0392	0.4142	PASS

Looking at data, the main point is that Poisson property is preserved at system input, but it is not preserved uniformly once requests move through internal services. This is expected in a

real microservice topology, as internal arrivals are affected by routing, parallel calls, processing delays, synchronization, congestion, and failed requests. A key limitation we have, is that due to the lack of a better arrival timestamp, IATs are computed from Jaeger span start times. Therefore, they represent when a service starts processing a request, not necessarily exact instant when request first arrives at service. This approximation is acceptable at low load, where arrival and processing start are close, but becomes more problematic at high load, where waiting & saturation separate the 2.

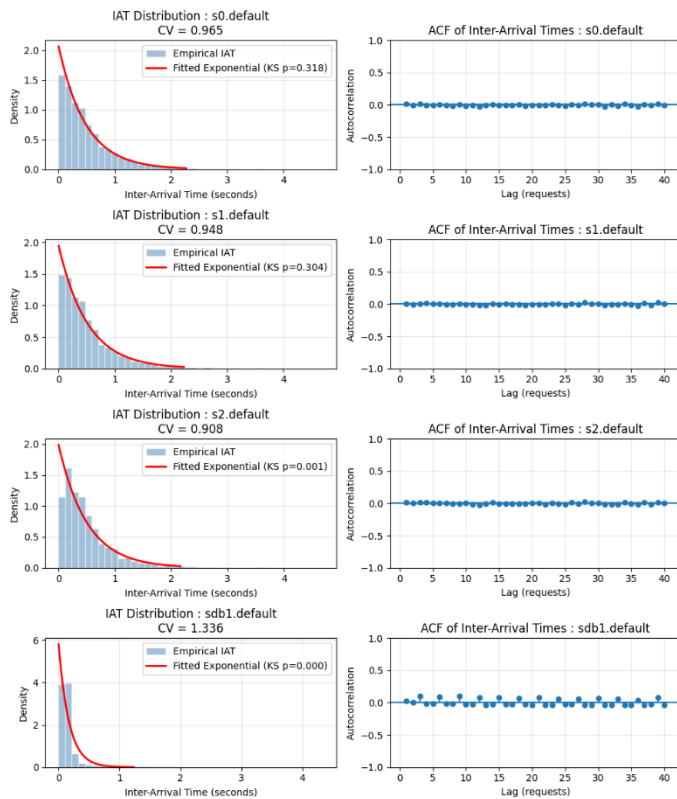
For s0, low load result behaves as expected. It passes KS test & CV is close to 1, meaning stream observed at s0 still resembles Poisson input produced by Runner. Since system is lightly loaded, there is little queueing distortion and span start times are a reasonable approximation of real arrivals. At medium & high load, however, this agreement disappears. KS fails & at high load the CV becomes much larger than 1, showing that the observed IATs are much more spread out than an exponential distribution would suggest. Conceptually, this indicates that arrival pattern seen through Jaeger has become irregular & bursty. However, this should not be interpreted as pure external arrival behavior. At high load, many requests are rejected and span starts no longer necessarily match true arrival instants, so the observed pattern also reflects congestion, scheduling, and the fact that only successfully started spans are being measured.

Both s1 & s2, move away from Poisson-like behavior as load increases. At low load, s1 still passes the KS test, while s2 already fails despite having a CV not extremely far from 1. This suggests that even at low load, the downstream structure is already slightly reshaping the arrival stream. At medium & high load, both services fail clearly, and their CV values fall below 1, so more deterministic IATs that appear more regular and more evenly spaced. Here we have s1 acting like a regulator, when the system becomes busy, requests are released downstream according to service completions rather than according to the original random Poisson input. Since s2 is called directly through s1, their behavior is naturally coupled, which explains why the two services show very similar patterns.

For sdb1, result is different as it receives calls from multiple upstream services rather than from a single path. At low load, sdb1 fails the KS test and has CV above 1, meaning the observed arrivals are more clustered than an exponential process. This is possible as sdb1 receives calls from different points in request path, 1 call comes directly from s0, while others arrive only after s1 and s2 perform their processing. Therefore, arrivals at sdb1 can occur in small groups rather than as one clean independent stream. At medium & high load, sdb1 instead passes the KS test & its CV is close to 1. This can be interpreted as a superposition effect, the combination of several upstream streams can make the aggregate stream look more Poisson like, even if each individual stream is not perfectly Poisson, a result supported by the Palm–Khintchine theorem^[3]

LOW LOAD

Assumption 1 : Poisson Arrivals at All Services



At low load, s0 & s1 behave closest to expected Poisson arrival assumption. Their IAT histograms have the typical exponential shape.

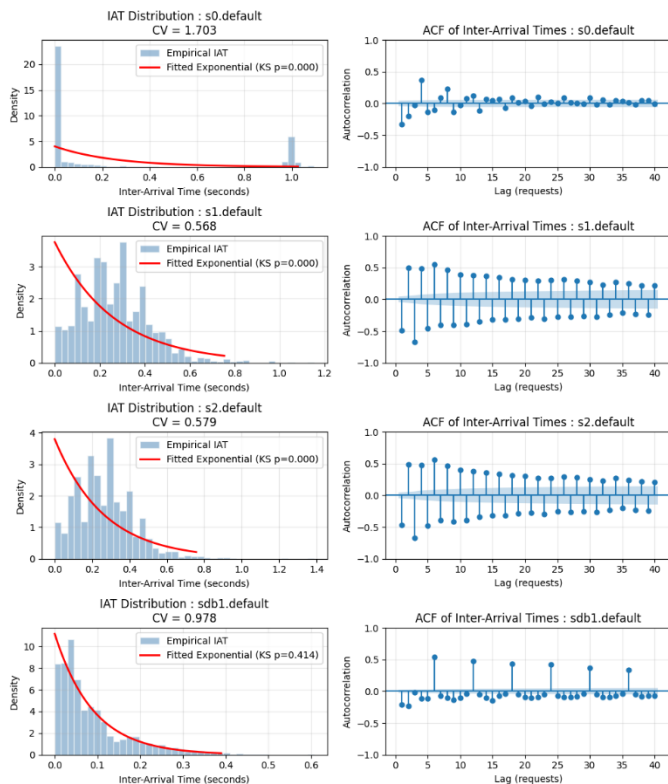
s2 roughly still has an exponential shape, but fit is weaker, with less variability.

sdb1's histogram is very concentrated near 0. This is consistent with sdb1 receiving calls from multiple upstream services, so several arrivals can occur very close together.

ACFs are all close to 0 for all services, meaning there is no strong temporal dependence between consecutive IATs. For s0, s1, and s2, the dots stay almost perfectly around 0. For sdb1, there are slightly larger fluctuations, but still no sustained autocorrelation pattern.

HIGH LOAD

Assumption 1 : Poisson Arrivals at All Services



At high load, s0 shows a highly distorted, two-cluster IAT distribution rather than an exponential one. Most observed IATs are almost zero & a 2nd visible concentration appears around 1 second, resulting in a very high CV of 1.703. ACF further shows a non-random structure at the first lags, indicating dependence.

CONSIDERATION: Due to lack of a better arrival timestamp, IATs are computed from Jaeger span start times. Therefore, we are not measuring the exact time when a request arrives at the service, but time when the request starts being processed by the service. Limitations of the approach are visible at high load, when request arrival and request processing no longer coincide.

s1 & s2's histograms also clearly fail to resemble exponential distributions. IATs are concentrated around a more regular spacing, with CV values around 0.57, meaning they are less variable than exponential and more deterministic like. This is expected because s1 and s2 receive traffic only after upstream processing, so the original Poisson input has been shaped by service execution, queueing & synchronization effects. ACF also shows autocorrelation across many lags, indicating presence of memory.

sdb1's histogram is closest to exponential. It is right-skewed, has high density near zero, then decays to the right. This is a clear illustration of the superposition effect, where when multiple upstream streams are merged their aggregate resembles a Poisson process. ACF shows some isolated spikes, so it is not perfectly independent, but it lacks the sustained oscillatory seen in s1 and s2.

3.2. Assumption 2: Exponential Service Times

We have seen that classical queueing theory assumes that the time a server spends processing each request follows an exponential distribution.

So, the remaining service time of a request in progress is independent of how long it has already been served. This implies that CV of service times equals exactly 1, and that successive service times are independent of one another.

In this section, we aim to verify whether this assumption holds in practice within our microservice setup.

3.2.1. Internal Service-Time Distribution

In this part of the notebook, we test whether the time each service spends on its own local computation (excluding downstream calls), follows an exponential distribution. This is the purest measure of the service time in the theoretic sense, as it isolates the work performed by the service itself.

Jaeger span durations are not suitable for this purpose, since they capture the full time, a span is open, including any blocking wait for downstream responses. Instead, internal service times are reconstructed from the Prometheus `mub_internal_processing_latency_milliseconds` histogram using the `reconstruct_samples_from_histogram_buckets` helper described earlier.

At a high level, in the code we:

- Iterate over all services and retrieve mean internal latency from Prometheus using the `prometheus_mean` helper.
- Reconstruct an approximate flat array of individual internal latency samples from the Prometheus histogram buckets using `reconstruct_samples_from_histogram_buckets`.
- Compute CV of reconstructed samples as a first-order check.
- Fit an exponential distribution to the reconstructed samples and apply a KS test on a capped sample as a second-order check.

We obtain the following results (across all loads):

<i>SERVICE</i>	MEAN INTERNAL LATENCY	SERVICE RATE (REQ/S)	CV	KS STATISTIC	KS P-VALUE	VERDICT
LOW LOAD						
<i>s0</i>	0.359	2783.88	2.514	0.4810	0.0000	FAIL
<i>s1</i>	289.069	3.459	0.593	0.6321	0.0000	FAIL
<i>s2</i>	280.391	3.567	0.353	0.6321	0.0000	FAIL
<i>sdb1</i>	16.474	60.703	1.329	0.5830	0.0000	FAIL
MEDIUM LOAD						
<i>s0</i>	0.892	1120.63	3.144	0.3807	0.0000	FAIL
<i>s1</i>	408.364	2.449	0.435	0.6207	0.0000	FAIL
<i>s2</i>	354.206	2.823	0.282	0.6146	0.0000	FAIL
<i>sdb1</i>	19.166	52.175	0.330	0.6308	0.0000	FAIL
HIGH LOAD						
<i>s0</i>	1.166	857.66	2.154	0.3663	0.0000	FAIL
<i>s1</i>	518.898	1.927	0.367	0.6191	0.0000	FAIL
<i>s2</i>	466.440	2.144	0.626	0.6256	0.0000	FAIL
<i>sdb1</i>	22.327	44.789	0.468	0.6308	0.0000	FAIL

Looking at data, the exponential service time assumption fails for every service, at every load level. All p-values are zero, so the empirical internal service-time distributions are not compatible with an exponential model.

For *s0*, internal latency is extremely small compared to other services, as it performs almost no real computation locally. However, its CV is always far above 1, so its internal service

time is highly variable relative to its mean. This does not mean s_0 is computationally expensive, rather even small runtime effects such as scheduling, container overhead, or occasional execution delays can create large relative variability.

s_1 & s_2 's pattern is the opposite. Their CV values are consistently below 1, meaning their internal service times are less variable than an exponential distribution. This is due to their execution of fixed artificial computation, producing a processing time that is relatively regular and close to deterministic. Their service rate also decreases as load increases, showing that service time is not fully load independent, likely because concurrent workers compete for CPU or memory resources.

For sdb_1 , low load shows higher variability, with CV above 1, which is plausible for disk I/O because filesystem and OS behavior can introduce irregular delays. However, it still fails the KS test, so matching the CV roughly is not enough to claim exponential service time. At medium and high load, its CV drops below 1, meaning the disk work becomes more regular. This suggests that under higher activity, I/O operations become more serialized or consistently scheduled.

3.2.2. Sojourn-Time Distribution per Service

In this section of the notebook, we consider the full sojourn time at each service (own computation + downstream calls).

We test its distribution to provide additional insight into the overall system behavior & see how the inclusion of downstream calls into the calculations, may make the overall results differ significantly from internal service time ones.

At a high level, in the code we:

- For each service we extract span durations from Jaeger data we previously collected.
- Compute descriptive statistics including mean, std, CV etc.
- Fit exponential distribution and apply KS test on a capped sample to assess whether the sojourn-time distribution is compatible with exponential model.

We obtain the following results (across all loads):

<i>SERVICE</i>	MEAN	STD	CV	P95	MAX	KS STATISTIC	KS P- VALUE	VERDICT
LOW LOAD								
<i>s0.default</i>	645.175	364.257	0.565	1358.448	3942.181	0.4238	0.0000	FAIL
<i>s1.default</i>	603.726	276.310	0.458	1208.520	2010.980	0.4336	0.0000	FAIL
<i>s2.default</i>	306.031	141.557	0.463	617.603	1079.205	0.4397	0.0000	FAIL
<i>sdb1.default</i>	18.348	9.303	0.507	31.919	409.323	0.4164	0.0000	FAIL
MEDIUM LOAD								
<i>s0.default</i>	10250.069	5584.734	0.545	20742.927	23224.851	0.2971	0.0000	FAIL
<i>s1.default</i>	859.338	126.863	0.148	1092.678	1503.770	0.5038	0.0000	FAIL
<i>s2.default</i>	410.967	113.175	0.275	612.490	864.696	0.4300	0.0000	FAIL
<i>sdb1.default</i>	22.536	8.018	0.356	38.078	91.953	0.4278	0.0000	FAIL
HIGH LOAD								
<i>s0.default</i>	60176.871	16005.470	0.266	71738.811	73034.640	0.4729	0.0000	FAIL
<i>s1.default</i>	1064.226	198.972	0.187	1443.787	2240.138	0.4756	0.0000	FAIL
<i>s2.default</i>	514.930	158.584	0.308	783.699	1311.255	0.3900	0.0000	FAIL
<i>sdb1.default</i>	25.255	9.513	0.377	43.036	167.295	0.4088	0.0000	FAIL

Looking at the data, sojourn times likewise fail exponential assumption for every service and every load level. This is expected because sojourn time is not pure service time, it further includes any waiting for downstream calls. Therefore, for services such as s0, s1 & s2 the sojourn time distribution is shaped by the whole dependency chain, not only by the service's own computation.

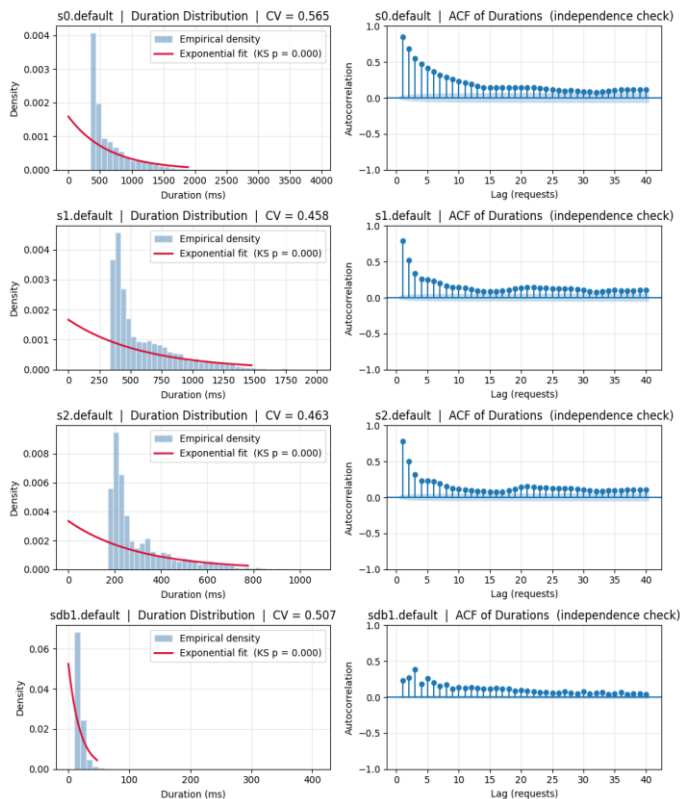
s0's most important observation is the huge growth in mean sojourn time as load increases. This does not mean s0 itself becomes computationally expensive, because its internal processing time is almost negligible. It means s0 remains blocked while waiting for downstream services, especially s1 and the rest of the chain. At high load, CV becomes very low and the p95 is close to the maximum, meaning most successful requests experience similarly long delays. So, system is not producing a wide mix of fast and slow successful requests; instead, successful requests are almost all delayed heavily.

For s1 & s2, sojourn times are also non-exponential and have CV values below 1. This fits the previous service time analysis, as both services perform relatively regular artificial computation, so their timing is much less random than an exponential distribution. Their mean sojourn time grows with load, showing that downstream waiting and resource contention become more relevant.

For sdb1, interpretation is simpler as it has no downstream calls. Hence its sojourn time & internal service correspond with one another.

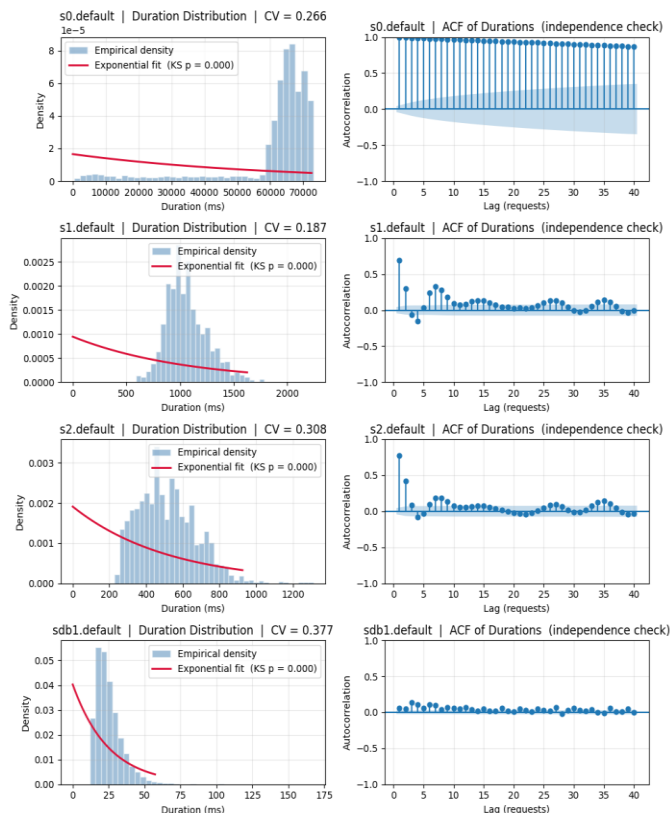
LOW LOAD

Sojourn Time Analysis — All Services



HIGH LOAD

Sojourn Time Analysis — All Services



All 4 services show right skewed duration distributions, meaning most requests complete in a relatively small time, while a smaller number of requests take longer.

s0's durations are largest as it is the entry service & its span includes waiting for downstream services. ACF shows strong positive autocorrelation that slowly decreases, meaning consecutive requests tend to have related durations. Even at low load, downstream waiting introduces dependency between requests.

s1 & s2 have similar shapes, with most durations concentrated in a narrower range & their ACF plots show positive autocorrelation as well.

sdb1 has smallest durations, mostly concentrated below a few tens of milliseconds, as it has no downstream calls and its sojourn time mainly reflects own disk I/O work. ACF also shows dependence between consecutive disk operations.

s0's histogram shows that the system is so saturated that nearly every request experiences a near-worst-case wait, which is why the CV dropped to 0.266. The ACF, likewise shows persistent positive autocorrelation across all lags, suggesting strong dependence between requests.

s1 & s2 show almost bell shaped histograms, reflecting the fact that sojourn time is now dominated by a relatively stable sum of deterministic internal computation plus a fairly consistent queueing wait. ACFs show some positive correlation at first lags, but this correlation does not remain as strong as in s0. This means nearby requests are somewhat related, probably as they pass through same workers & queueing conditions, but the dependency is not as persistent.

sdb1's histogram remains right-skewed & concentrated near zero, looking visually similar to low load. Its sojourn time mostly reflects its own local disk work. ACF values stay close to 0, so consecutive sdb1 durations are almost independent.

This further supports our previous claim, at high load, the 3 upstream streams overlap much more densely, smoothing out individual caller patterns and making consecutive sdb1 durations appear approximately independent.

3.2.3. Service-Time Decomposition: Internal vs External

This last section for assumption, does not constitute a formal test, rather, it serves for an interpretive role, decomposing the mean total request time at each service into its two components (internal computation & downstream calls).

This decomposition provides the context needed to understand the results of the two preceding subsections. A service whose sojourn-time distribution deviates strongly from exponential may do so simply because it is dominated by downstream waiting, which introduces additional variability & structure that has nothing to do with the service's own processing characteristics.

At a high level, in the code we:

- For each service, retrieve the mean internal, external & total request latency from Prometheus.
- Compute the percentage contribution of internal and external time to the total mean sojourn time.

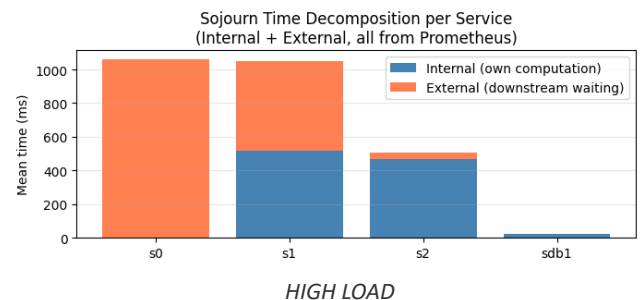
We obtain the following results (across all loads):

<i>Service</i>	W Total (ms)	W Internal (ms)	Internal %	W External (ms)	External %
LOW LOAD					
<i>s0</i>	610.159	0.359	0.1%	609.546	99.9%
<i>s1</i>	604.844	289.069	47.8%	315.398	52.1%
<i>s2</i>	305.085	280.391	91.9%	24.327	8.0%
<i>sdb1</i>	16.818	16.474	98.0%	0.071	0.4%
MEDIUM LOAD					
<i>s0</i>	823.134	0.892	0.1%	822.058	99.9%
<i>s1</i>	812.706	408.364	50.2%	403.829	49.7%
<i>s2</i>	384.899	354.206	92.0%	30.176	7.8%
<i>sdb1</i>	19.556	19.166	98.0%	0.079	0.4%
HIGH LOAD					

<i>s0</i>	1062.082	1.166	0.1%	1060.556	99.9%
<i>s1</i>	1049.068	518.898	49.5%	529.435	50.5%
<i>s2</i>	505.407	466.440	92.3%	38.238	7.6%
<i>sdb1</i>	22.798	22.327	97.9%	0.093	0.4%

Looking at the data, we can make the following observations:

- *s0*
 - As we expect, *s0* is almost entirely downstream bound at every load level, with external time accounting for 99.9% of its total sojourn time. The decomposition explains directly why the sojourn time distribution at *s0* will be shaped almost entirely by whatever distribution governs the downstream response times, rather than by *s0*'s own processing.
- *s1*
 - We observe an almost 50%-50% split between internal & external wait. Sojourn is not dominated by any single highly variable component, with both halves being relatively concentrated distributions.
- *s2*
 - Dominated by its own internal computation, which accounts for approximately ~90% of total sojourn, meaning its sojourn is almost entirely determined by its internal processing time.
- *sdb1*
 - Only internal computation. Sojourn is equal to internal work done.



3.3. Assumption 3: Single Server Behavior

This our third main assumption that we test. As we know, in classical queueing theory in its simplest form, we assume that each service node is attended by a single server, giving rise to the M/M/1 model. This assumption implies that requests are processed one at a time, with no parallelism at the node level. In practice however, services are often configured with multiple concurrent workers, which changes the queueing model that should be applied. In this section we verify whether the single-server assumption holds for the services in our topology.

<i>METRIC</i>	<i>VALUE</i>
<i>Workers per service</i>	4
<i>Verdict</i>	FAIL (not single-server)
<i>Correct model</i>	M/M/c with $c = 4$

The same result is obtained across all load levels, which is expected given that we are not changing our topology across runs. The server multiplicity is a static configuration property of the deployment rather than a function of the offered load.

The failure here is by design and fully expected. As established in the topology description, each service in our μ Bench deployment is configured with 4 workers, meaning each node can process up to 4 requests concurrently. This directly violates the single-server assumption of M/M/1. The correct analytical model for each node is therefore M/M/4, so $c = 4$.

3.4. Assumption 4: Infinite Buffer

The 4th assumption is infinite buffer. In classical queueing models, we assume an infinite buffer, meaning the system can accommodate an unbounded number of waiting requests without dropping any. In practice, real systems impose finite queue limits, finite resources, and once these are exhausted, incoming requests are rejected. In this section we use the HTTP status codes recorded by the Runner to assess whether any requests were dropped, treating a non-200 response as a proxy for buffer exhaustion or request rejection at the service level.

<i>LOAD LEVEL</i>	<i>TOTAL REQUESTS</i>	<i>NON-200 RESPONSES</i>	<i>VERDICT</i>
<i>Low</i>	5000	0	PASS
<i>Medium</i>	5000	0	PASS
<i>High</i>	5000	2811	FAIL (56.22% non-200)

At low & medium load the system handles all 5000 requests successfully, with zero rejections. This is consistent with both being operating points where the system is not saturated. The infinite buffer assumption therefore holds at these load levels.

On the other hand, at high load the picture changes. Over half of all dispatched requests, receive a non-200 response, indicating that the system is actively rejecting requests. This is a direct consequence of saturation. At a mean IAT of 100ms the offered load exceeds what the system can sustain, internal queues overflow, and the system begins returning errors. Thus, the infinite buffer assumption fails conclusively at high load.

3.5. Further Considerations

3.5.1. Little's Law

Little's Law is a fundamental result in queueing theory that relates 3 quantities of a stable system:

$$L = \lambda * W$$

where L is the mean number of requests present in the system, λ is the throughput (mean arrival rate of completed requests), and W is the mean time a request spends in the system.

The uniqueness of the law is that it is remarkably general, so it holds for any stable queueing system regardless of the arrival distribution, service time distribution, or number of servers, requiring only that the system is in steady state.

In this section we combine the mean sojourn time (W) and the completed throughput (λ) observed at each service, to compute L , the mean number of requests simultaneously present at each node. This gives us a concrete characterization of how loaded each service is under the 3 load conditions.

At a high level, in the code we:

- Extract spans from Jaeger data for each service.
- Compute the mean sojourn time W , as we previously did.
- Compute throughput (X) as the total number of completed spans divided by the observation window.
- Apply Little's Law to obtain $L = X \cdot W$, the mean number of requests in the system at each service.

<i>SERVICE</i>	<i>W (s)</i>	<i>X (req/s)</i>	<i>L = X * W</i>
LOW LOAD			
<i>s0</i>	0.6452	1.9965	1.2881
<i>s1</i>	0.6037	1.9965	1.2053
<i>s2</i>	0.3060	1.9967	0.6111
<i>sdb1</i>	0.0183	5.9895	0.1099
MEDIUM LOAD			
<i>s0</i>	10.2501	4.8294	49.5021
<i>s1</i>	0.8593	4.8364	4.1561
<i>s2</i>	0.4110	4.8373	1.9880
<i>sdb1</i>	0.0225	14.5093	0.3270
HIGH LOAD			
<i>s0</i>	60.1769	3.7542	225.9148
<i>s1</i>	1.0642	3.7099	3.9482
<i>s2</i>	0.5149	3.7112	1.9110
<i>sdb1</i>	0.0253	11.1308	0.2811

Here, X is the completed throughput observed from Jaeger spans & W is mean span duration. Hence, L represents mean number of open spans in service.

sdb1 remains low in all load conditions. Even though it receives the highest throughput, because it is visited 3 times per external request, its mean duration is very small.

s1 & *s2* increase from low to medium load, showing that more requests are simultaneously present as workload increases. However, their L values do not explode at high load. This at first is counter intuitive, but it is due to the system rejecting requests, so overall completed throughput decreases. As a result, L is limited as fewer requests successfully complete & appear in span data.

s0's L grows up to 225.9 at high load. This is not due to *s0*'s own local computation, as it has almost no internal work, but its span remains open while it waits for downstream services. Therefore, the huge Lat *s0* mainly represents many requests waiting for the downstream chain to finish. In other words, Little's Law at *s0* captures propagated downstream congestion, not local queue buildup inside *s0*.

3.5.2. Visit Ratio Validation

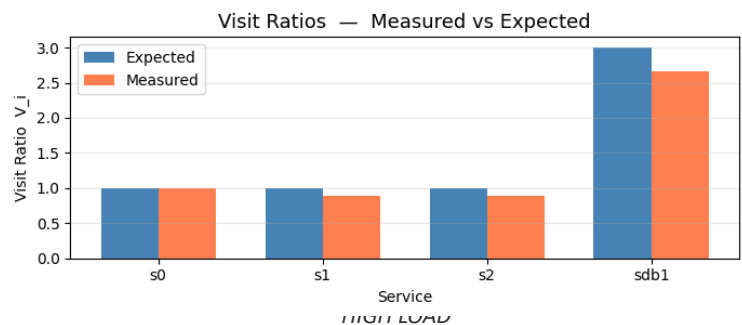
The visit ratio V_i of a service is defined as the mean number of times that service is visited per external request entering the system. Under the topology defined in our workmodel, the expected visit ratios are deterministic, so s_0 , s_1 , and s_2 each receive exactly one visit per external request, while $sdb1$ receives three.

Here, we validate whether the routing behavior observed empirically in the Jaeger traces matches these topology-implied expectations.

SERVICE	SPANS	MEASURED V	EXPECTED V	DEVIATION
LOW & MEDIUM LOAD				
<i>s0.default</i>	5000	1.000	1.000	+0.00%
<i>s1.default</i>	5000	1.000	1.000	+0.00%
<i>s2.default</i>	5000	1.000	1.000	+0.00%
<i>sdb1.default</i>	15000	3.000	3.000	+0.00%
HIGH LOAD				
<i>s0.default</i>	2176	1.000	1.000	+0.00%
<i>s1.default</i>	1935	0.889	1.000	-11.08%
<i>s2.default</i>	1933	0.888	1.000	-11.17%
<i>sdb1.default</i>	5805	2.668	3.000	-11.08%

At low & medium loads measured visit ratios match the expected topology exactly, with 0 deviation at every service. This confirms that the routing logic is functioning correctly and that every request successfully propagates through the full call graph as designed.

At high load a deviation of approximately -11% appears at s_1 , s_2 , and $sdb1$, while s_0 remains at exactly 1 by definition. This result is expected & it is a direct consequence of the buffer exhaustion finding established in Assumption 4. At high load, 56.22% of requests dispatched to the system received non-200 responses.



Those failed requests were rejected at or before s_0 could successfully forward them downstream, meaning a portion of requests that registered a span at s_0 never produced a corresponding span at s_1 , s_2 , or $sdb1$.

3.5.3. Bottleneck Identification

A bottleneck of a system is the node with the highest utilization, so the node that is closest to exceeding its maximum throughput capacity. In the case of an M/M/1 queue, utilization would be defined like this:

$$U_i = \frac{\lambda_i}{\mu_i}$$

where λ_i is the arrival rate at service i and μ_i is the service rate of the single server. However, as established in our topology, each service is configured with 4 workers, so requests are served in parallel across 4 concurrent servers. Thus, the effective service capacity of a node is $c \cdot \mu_i$ rather than μ_i alone. The formula thus becomes:

$$U_i = \frac{\lambda_i}{c \cdot \mu_i}$$

So, in this section we compute utilization for each of our services by combining three sources:

- External arrival rate from the Runner's result text file.
- Visit ratios as we previously computed.
- Mean internal processing times as we previously computed. (internal work only)

SERVICE	V_i	λ_i	S_i	U_i
LOW LOAD				
s_0	1.000	1.996	0.359	0.0002
s_1	1.000	1.996	289.069	0.1443
s_2	1.000	1.996	280.391	0.1399
$sdb1$	3.000	5.989	16.474	0.0247
MEDIUM LOAD				

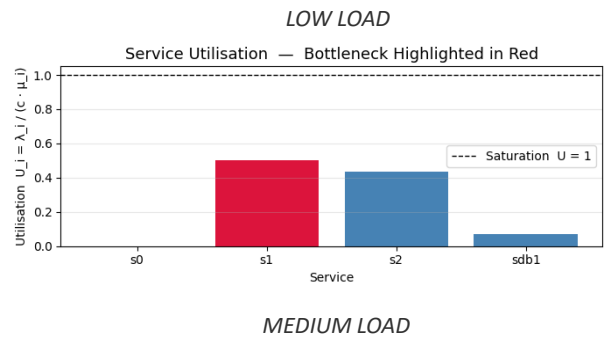
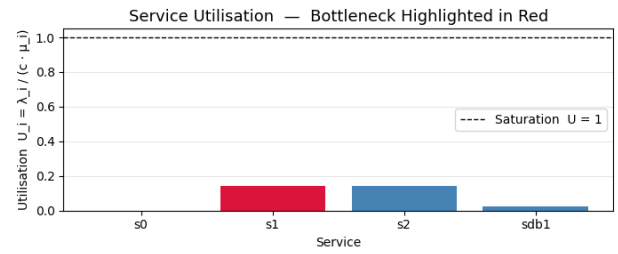
<i>s0</i>	1.000	4.926	0.892	0.0011
<i>s1</i>	1.000	4.926	408.364	0.5029
<i>s2</i>	1.000	4.926	354.206	0.4362
<i>sdb1</i>	3.000	14.777	19.166	0.0708
HIGH LOAD				
<i>s0</i>	1.000	10.101	1.166	0.0029
<i>s1</i>	0.889	8.982	518.898	1.1652
<i>s2</i>	0.888	8.973	466.440	1.0463
<i>sdb1</i>	2.668	26.947	22.327	0.1504

	LOW LOAD	MEDIUM LOAD	HIGH LOAD
<i>Bottleneck</i>	s1	s1	s1
<i>Utilization</i>	0.144	0.503	1.165
<i>Status</i>	Not saturated	Not saturated	Saturated

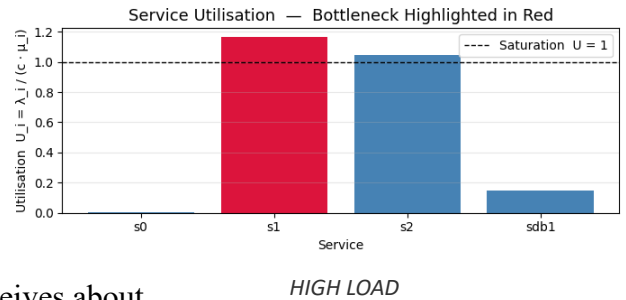
We calculate utilization using planned arrival rate, rather than effective. This decision will allow us to successfully evaluate whether offered workload exceeds service capacity of each node. Instead, if we used the effective arrival rate, at high load the utilization could appear lower, possibly even lower than medium load, due to the requests that are rejected & therefore disappear from the completed flow measurement.

At low & medium load, all services remain below saturation. s1 has highest utilization in both cases, reaching 0.144 at low load and 0.503 at medium load, so it is already the closest service to capacity.

At high load, the situation changes, with s1 reaching utilization 1.165 & s2 reaching 1.046, so both services require more capacity than their 4 workers can provide. Since s1 has the highest utilization across all loads, it is the bottleneck of the system.



s0 is never the bottleneck, its utilization remains almost zero because its own internal service demand is extremely small. Therefore, the large delays observed at s0 in Little's Law do not represent local congestion inside s0, rather are caused by downstream waiting.



sdb1 is also not a bottleneck. Even though it receives about 3 visits per external arrival, its service time is very small, so its utilization remains low even at high load. This means that the shared database-like service receives the largest number of calls, but it is fast enough not to limit the system.

A further important point is that service demand is not constant. In a classical M/M/c model, service time is assumed to be a fixed property of the server and independent of load. In the measurements, however, s1 grows from 289 ms at low load to 519 ms at high load, and s2 grows from 280 ms to 466 ms. This means that services themselves become slower as load increases, probably due to resource contention between concurrent workers. Therefore, the system deviates from the ideal Markovian queueing model not only because requests are rejected, but also because service rates degrade under load.

3.5.4. E2E Analysis

In this final section of our analysis, we take a black-box perspective on the entire microservice, treating it as a single E2E entity rather than examining individual services in isolation. This complements the previous analysis by capturing the aggregate effect of all internal queueing, processing, and downstream waiting as experienced by an external client.

At a high level, in the code we:

- Compute offered arrival rate λ directly from the Runner.
- Derive system service rate μ from the bottleneck service (s1).
 - This is best approximation available given the observability constraints of the stack. If we use Jaeger, span durations capture sojourn time rather than pure service time.
 - Only instrumentation thus is Prometheus. This metric is consequently the sole reliable source from which a meaningful service rate can be derived, and its mean value at s1 is what we use here.
 - This is therefore, not a directly observed E2E service rate, but a bottleneck-based estimate of the maximum sustainable system throughput.
- Compute $\rho = \lambda / \mu$ as a measure of system load.

- Compute E2E statistics.
- Apply Little's Law.

<i>METRIC</i>	LOW LOAD	MEDIUM LOAD	HIGH LOAD
λ (req/s)	1.996	4.926	10.101
μ (req/s)	13.838	9.795	7.709
$\rho = \lambda/\mu$	0.1443	0.5029	1.3104
<i>Succeeded</i>	5000/5000 (100%)	5000/5000 (100%)	2189/5000 (43.8%)
<i>Goodput X</i> (req/s)	1.997	4.832	3.759
<i>Mean E2E latency W</i>	0.676 s	9.799 s	85.458 s
<i>CV</i>	0.552	0.542	0.322
<i>p95</i>	1.405 s	19.771 s	106.674 s
<i>KS p-value</i>	0.0000 (FAIL)	0.0000 (FAIL)	0.0000 (FAIL)
$L = X \cdot W$	1.35	47.35	321.26

Results are broadly consistent with the per-service findings. The E2E analysis confirms the same global behavior observed earlier, so low & medium loads remain manageable, while high load exceeds capacity of the bottleneck and causes request dropping.

CV remains below 1 across all loads and decreases further under saturation, which mirrors sojourn-time behavior of s_0 , suggesting E2E latency is not exponentially distributed, but is dominated by the relatively regular processing and waiting pattern created by s_1 and s_2 under load.

A key point is that section, is that we combine 2 different views of the system. Arrival rate (λ) and load ratio (ρ) are based on offered workload by Runner. Like in utilization, this is necessary to evaluate whether workload imposed on system exceeds the estimated bottleneck capacity. Instead, goodput, response time, and Little's Law are computed only on successful completed requests, as failed requests do not provide a meaningful successful response time. Therefore, λ describes pressure applied to the system, while X , W & L describe flow that actually completed successfully.

4. CONCLUSION

Through this project we investigated how well classical queueing assumptions describe a real microservice system under low, medium, and high load. The main result is that the assumptions are useful as a starting point, but they become increasingly inaccurate once the internal topology, downstream calls, parallel workers, and saturation effects start shaping the observed behavior.

At the system input, Poisson arrival assumption holds as expected. The workload generated by the Traffic Generator and replayed by the Runner preserves exponentially distributed and independent inter-arrival times. However, once requests start to move through the internal services, this property is no longer preserved, rather these arrivals are reshaped by routing, service completions, synchronization & congestion.

The exponential service time assumption fails more clearly. Internal service times are rejected as exponential for every service and every load level. *s0* has extremely small local computation, so its measurements are dominated by runtime noise and occasional outliers. *s1* and *s2* perform fixed artificial computation, making their service times closer to deterministic than exponential. *sdb1* is shaped by local disk I/O behavior, which also does not match the memoryless exponential model. Therefore, the system does not behave like a Markovian queue with exponential service at each node.

The sojourn-time analysis further supports this & reaches the same conclusion from a broader perspective. Span durations are not just local service times, but they also include downstream waiting, which is especially important for *s0*, whose local work is negligible but whose span remains open while it waits for downstream services. As load increases, *s0*'s sojourn time grows massively because downstream congestion is reflected back to the entry service. Meanwhile *s1* & *s2*'s sojourn times remain non-exponential as they combine regular computation with waiting effects.

The structural assumptions also only partially hold. The single-server assumption fails by design, because each service has 4 workers, making $M/M/4$ a better structural model than $M/M/1$. The infinite-buffer assumption holds at low & medium load, but fails at high load, where a large fraction of requests receives non-200 responses.

Overall, classical queueing theory gives a useful language for interpreting systems, but the real microservices often do not fully satisfy the assumptions required by simple Markovian models. The input workload can be Poisson, but the internal system behavior may be shaped by a plethora of things. Therefore, the main conclusion is not that queueing theory is useless, but that simple models such as $M/M/1$ or ideal Jackson networks are only rough approximations, especially once the system approaches saturation.

5. REFERENCES

- [1] <https://github.com/mSvcBench/muBench>
- [2] <https://github.com/mSvcBench/muBench/blob/main/Docs/Manual.md#monitoring-and-tracing>
- [3] [https://www.wikiwand.com/en/Palm%E2%80%93Khintchine theorem](https://www.wikiwand.com/en/Palm%E2%80%93Khintchine_theorem)
- [4] [https://www.wikiwand.com/en/Little%27s law](https://www.wikiwand.com/en/Little%27s_law)
- [5] [https://www.wikiwand.com/en/M/M/c queue](https://www.wikiwand.com/en/M/M/c_queue)